```
Output:
I am everywhere
I am also everywhere, but did you get my name right?
Fatal error: Uncaught Error: Call to undefined function greetings()
```

First, we enter `Other` namespace and define a string variable `$val`, and a function `greetings()`. Next, we switch to root namespace and use `$val` in a `print` expression. The expression is valid because although `$val` was defined in a namespace, the scope was still global. After the `print` expression, we then call `greetings()`, fully qualified in the first instance and with no namespace qualification in the second. The second call fails because there's no `greetings()` defined in root namespace but here's the thing: when we defined `greetings()` in `Other` namespace it also went into global scope, but its true name became `\Other\greetings()`. Namespaced functions still have global scope, they just have more complex names.

All of the keywords we've used for interacting with namespaces are simply syntactic sugar (mmm, delicious) to avoid needing to pseudo-namespace our functions. Before PHP 5.3 our namespaced `collision()` functions would have been named `collision()` (root namespace), `Other_collision()` (`Other` namespace), `Other_Sub_collision()` and so on. The PSR-1 coding standard actually recommends using pseudo-namespaced function and class names for code targeting PHP 5.2 or earlier, as well as insisting on namespaced code for PHP 5.3 and later.

Functional Programming

It's quite likely that you will have at least heard the phrase "functional programming" (FP) at some point in your journey as a software engineer. It is often used in opposition to object-oriented programming (OOP), and I've endured many protracted arguments over the relative merits of these paradigms (and by extension which language is "the best") during my career. Which is exactly as much fun as it sounds. An advocate for the FP style would invariably point the finger at PHP and tut, for it was just another OOP language and filled with mutability and side effects. An OOP champion might then criticize FP for its tendency to allow programs to collapse into callback-hell singularities. Hilarity would often ensue shortly afterwards.